Módulo: MF0492_3..Unidad Formativa: UF1844 Desarrollo de Aplicaciones Web en el Entorno Servidor. Examen intermedio.

APELLIDOS y NOMBRE: VISÚS GARCÍA, VÍCTOR

FECHA: 10 DE ABRIL DE 2026

Se aportará el fichero con las capturas de pantalla pertinentes en el fichero **UF1844_APELLIDO1_NOMBRE.pdf** y **también los programas .py** que realizan las tareas. Si es necesario el uso de rutas de archivo en los programas, estas deben ser relativas, no absolutas.

1. Programar en Python una función tal que pasado como argumento una cadena de caracteres (tipo string) que represente un número en base2 (código binario), devuelva el valor decimal correspondiente a dicho número (como un número entero, tipo int). Así, por ejemplo, si se pasa "1101" debe retornar 13. Se atenderá a la modularidad, comentarios y legibilidad de la solución presentada. (3 puntos)

```
# Para el binario 100111, el cálculo es 1*2^5 + 0*2^4 + 0*2^3 + 1*2^2 + 1*2^1
+ 1*2^0 = 32+0+0+4+2+1=39.


"""
Convierte un número binario a decimal.
Parámetros:
numBinario (str): número binario a convertir.
Returns:
int: número decimal correspondiente al resultado de convertir el número
binario a decimal.
Ejemplo:
binary_to_decimal("100111") -> 39
"""


def conversorBinarioDecimal(numBinario):
    resultado = 0
    for i in range(len(numBinario)):
        resultado += int(numBinario[i]) * (2 ** (len(numBinario) - 1 - i))
    return resultado

    # print(binary_to_decimal("1101"))


"""
Solicita datos al usuario, y evalua que sea un número binario, si no es
binario lanza un error.
Returns: string: número binario introducido por el usuario
"""


def solicitaDatos():
    numero = input("Introduce un número binario: ")
```

```python
  for i in range(len(numero)):
    if numero[i] != "0" and numero[i] != "1":
      raise ValueError
    else:
      return numero

"""
El metodo principal del ejercicio
mientras que el usuario no introduzca un numero binario, sigue solicitandole
que intruduzca el dato
Imprime el resultado en consola
"""

def appIni():
  flag = True
  while flag:
    try:
      print(
        f"El numero en decimal es: {conversorBinarioDecimal(solicitaDatos())}"
      )
      flag = False
    except ValueError:
      print("El número introducido no es binario")

# Ejecuta el ejercicio
appIni()
```

```
Victor@NEUROMANTE MINGW64 /c/AppDesarrollo/python (main)
$ py 00_examen/UF1844_VISUS_VICTOR/UF1844-Ejercicio01_VISUS_VICTOR.py
Introduce un número binario: 11001
El numero en decimal es: 25

Victor@NEUROMANTE MINGW64 /c/AppDesarrollo/python (main)
$
```

2. Programar una función en Python que pida y tome del usuario un conjunto de 3 números enteros, con valores posibles entre 0 y 99, sin repeticiones

, programar también una segunda función en Python que, también tomando del usuario un conjunto de 3 números enteros, calcule y retorne todos los que faltan entre el mayor y el menor de los enteros dados anteriormente (no cuántos faltan, sino una lista con todos los que faltan). Tratar para la segunda de estas funciones las **siguientes excepciones**:

• Alguno de los números en el argumento de la función es menor que 0 o es mayor que 99.

• Entre los argumentos de la función hay valores que no representan un número entero.

Generar un diccionario con 4 claves y cambios dinámicos en los valores, cada una de las cuales indicará la amplitud de cada uno de los cuatro intervalos en que queda dividido el recorrido 0-99. A modo de ejemplo, si un usuario suministra: 5 45 66 el diccionario será:

{

“intervalo1”: 5

“intervalo2”: 40

“intervalo3”: 21

“intervalo4”: 34

}

Por último, transformar dicho diccionario a formato JSON.

Invocar las funciones necesarias para poder visualizar los resultados.

Probarlo con dos entradas diferentes de usuario.

(4 puntos)

```
"""

2. Programar una función en Python que pida y tome del usuario un conjunto de
3 números enteros, con valoresposibles entre 0 y 99, sin repeticiones,
programar también una segunda función en Python que, también tomando del
usuario un conjunto de 3 números enteros, calcule y retorne todos los que
faltan entre el mayor y el menor de los enteros dados anteriormente (no
cuántos faltan, sino una lista con todos los que faltan). Tratar para la
segunda de estas funciones las siguientes excepciones:
• Alguno de los números en el argumento de la función es menor que 0 o es
mayor que 99.
• Entre los argumentos de la función hay valores que no representan un número
entero.
Generar un diccionario con 4 claves y cambios dinámicos en los valores, cada
una de las cuales indicará la amplitud de cada uno de los cuatro intervalos en
que queda dividido el recorrido 0-99. A modo de ejemplo, si un usuario
suministra: 5 45 66 el diccionario será:
{
```

```
    “intervalo1”:  5
    “intervalo2”: 40
    “intervalo3”: 21
    “intervalo4”: 34
}
Por último, transformar dicho diccionario a formato JSON.
Invocar las funciones necesarias para poder visualizar los resultados.
Probarlo con dos entradas diferentes de usuario.
(4 puntos)
"""

import json

"""
Pide al usuario 3 números enteros y válidos, sin repeticiones, y los devuelve
en un conjunto.
Args: numRepeticiones (int): El número de repeticiones que tiene que hacer la
solicitud al usuario, por defecto 3
Returns: set: Un conjunto con los 3 números enteros y válidos.
"""


def solicitaNumeros(numRepeticiones=3):

  conjuntoNumeros = list()

  while len(conjuntoNumeros) < numRepeticiones:
    try:
      numero = int(input("Introduce un número entero: "))
      if validaNumeros(numero) and validarRepeticiones(numero,
conjuntoNumeros):
        conjuntoNumeros.append(numero)
    except ValueError as e:
      print(e)

  return conjuntoNumeros


"""
Valida si un número est  entre 0 y 99.
Args: numero (int): El número a validar.

Returns: bool: True si el número es válido, False en caso contrario.
"""


def validaNumeros(numero):

  if 0 <= numero <= 99:
    return True
```

```
    else:
        raise ValueError("El número introducido no es válido, debe estar entre 0 y
99")
        return False

"""
Valida si un número ya ha sido introducido anteriormente.
Args:
    numero (int): El número a validar.
    conjuntoNumeros (set): El conjunto de números introducidos anteriormente.
Returns: bool: True si el número no ha sido introducido anteriormente, False
en caso contrario.
"""

def validarRepeticiones(numero, conjuntoNumeros):

    if numero not in conjuntoNumeros:
        return True
    else:
        raise ValueError(f"El número {numero} ya ha sido introducido
anteriormente")
        return False

"""
Función en Python que, también tomando del usuario un conjunto de 3 números
enteros, calcule y retorne todos los que faltan entre el mayor y el menor de
los enteros dados anteriormente (no cuántos faltan, sino una lista con todos
los que faltan). Tratar para la segunda de estas funciones las siguientes
excepciones:
"""

"""
Haciendo nuso de la funcion solicitarNumeros(),
pide al usuario tantos numeros como repeticiones se indique en el argumento de
la funcion solicitarNumeros(), en este caso 3
Calcule y devuelve todos los que faltan entre el mayor y el menor de los
enteros dados
anteriormente (no cuántos faltan, sino una lista con todos los que faltan).

Returns: list: Una lista con todos los números que FALTAN entre el mayor y el
menor de los enteros dados.
"""

def faltantes():

    try:
```

```
    # declaro el conjunto lista con los 3 números, llamando la funcion
solicitaNumeros, que pide una serie de nnumeros al usuario
    conjuntoNumeros = solicitaNumeros(3)

    # creo una lista con todos los números que faltan entre el mayor y el
menor de los enteros dados
    numerosFaltantes = list(range(min(conjuntoNumeros) + 1,
max(conjuntoNumeros)))

    # recorro ambas listas comparando los numeros de cada una y los remuevo de
la lista de numerosFaltantes, eliminando el que coincidan con los de la lista
conjuntoNumeros
    for numero in conjuntoNumeros:
      if numero in numerosFaltantes:
        numerosFaltantes.remove(numero)
    return numerosFaltantes

  except ValueError as e:
    print("2--", e)  # lanza el error en la primera funcion solicitaNumeros()


def generarIntervalosYJson(lista):
  # 1. Ordenar la lista para calcular distancias
  n = sorted(lista)
  intervalos = {}

  # 2. Primer intervalo (desde 0 hasta el primer número)
  intervalos["intervalo1"] = n[0]

  # 3. Intervalos intermedios (distancia entre números)
  # Recorremos desde el segundo número hasta el último
  for i in range(1, len(n)):
    intervalos[f"intervalo{i + 1}"] = n[i] - n[i - 1]

  # 4. Último intervalo (desde el último número hasta 100)
  # La clave será "intervalo" + (cantidad de números + 1)
  intervalos[f"intervalo{len(n)}"] = 100 - n[-1]

  # 5. Transformar a JSON
  intervalos_json = json.dumps(intervalos, indent=4)

  return intervalos, intervalos_json


def appIni():
  print(
    "Ejercicio 2: Primera función: ----------------------------\n",
    solicitaNumeros(),
  )
```

```
    print("Ejercicio 2: Segunda función: -------------------------\n",
faltantes())

    print("Ejercicio 2: Tercera Parte: ----------------------------\n")

    # Invocamos tu función original
    mis_numeros = solicitaNumeros(4)

    print(f"\nNúmeros obtenidos: {mis_numeros}")

    # Calculamos intervalos y JSON
    diccionario, el_json = generarIntervalosYJson(mis_numeros)

    print("\nDiccionario de amplitudes:")
    print(diccionario)

    print("\nResultado en formato JSON:")
    print(el_json)


appIni()
```

```
Victor@NEUROMANTE MINGW64 /c/AppDesarrollo/python (main)
$ py 00_examen/UF1844_VISUS_VICTOR/UF1844-Ejercicio02_VISUS_VICTOR.py
Introduce un número entero: 1
Introduce un número entero: 5
Introduce un número entero: 10
Ejercicio 2: Primera función: ----------------------------
 [1, 5, 10]
Introduce un número entero: 10
Introduce un número entero: 20
Introduce un número entero: 30
Ejercicio 2: Segunda función: --------------------------
 [11, 12, 13, 14, 15, 16, 17, 18, 19, 21, 22, 23, 24, 25, 26, 27, 28, 29]
Ejercicio 2: Tercera Parte: ----------------------------

Introduce un número entero: 2
Introduce un número entero: 5
Introduce un número entero: 8
Introduce un número entero: 10

Números obtenidos: [2, 5, 8, 10]

Diccionario de amplitudes:
{'intervalo1': 2, 'intervalo2': 3, 'intervalo3': 3, 'intervalo4': 90}

Resultado en formato JSON:
{
    "intervalo1": 2,
    "intervalo2": 3,
    "intervalo3": 3,
    "intervalo4": 90
}
```

3. Escribe una función en Python llamada cuenta_palabras que reciba un parámetro (nombre de archivo) y cuente el número de palabras del archivo y las haga figurar como una nueva línea al final del mismo. Deben excluirse como palabra los grupos de caracteres que sean númericos con o sin signos (caracteres) de puntuación. Probarlo con el archivo que debe ser accedido desde tu directorio local (ruta relativa: ./ficheros/econom.txt).

que tiene este contenido:

En España, la evolución económica del siglo se divide en tres fases principales: la primera etapa (1800-1833) estuvo marcada por una grave crisis económica debido a la interrupción del comercio con América y las guerras de independencia; la segunda etapa (1834-1874) vio el surgimiento de focos industriales en Cataluña, aunque el crecimiento fue lento debido a la falta de capitales y conflictos entre proteccionismo e librecambismo; y la tercera etapa (1875-1914) aceleró el desarrollo con la creación de un sector bancario y la industria siderometalúrgica, aunque sin alcanzar el despegue total de otras potencias europeas.

de modo que al ejecutarse la función quede (la cursiva es opcional):

En España, la evolución económica del siglo se divide en tres fases principales: la primera etapa (1800-1833) estuvo marcada por una grave crisis económica debido a la interrupción del comercio con América y las guerras de independencia; la segunda etapa (1834-1874) vio el surgimiento de focos industriales en Cataluña, aunque el crecimiento fue lento debido a la falta de capitales y conflictos entre proteccionismo e librecambismo; y la tercera etapa (1875-1914) aceleró el desarrollo con la creación de un sector bancario y la industria siderometalúrgica, aunque sin alcanzar el despegue total de otras potencias europeas.

92 palabras

(4 puntos)

```
import re

"""
Función que cuenta las palabras en un archivo, excluyendo las que contienen
números,
y agrega el conteo al final del archivo.
"""

def cuentaPalabras(fichero):
  # Bloque try para manejar excepciones, como si el archivo no existe
  try:
    # Abrir el archivo en modo lectura para obtener su contenido
    with open(fichero, "r", encoding="utf-8") as f:
```

```
        contenido = f.read()

    # Usar expresiones regulares para encontrar todas las secuencias de
caracteres alfanuméricos (palabras)
    palabras = re.findall(r"\b\w+\b", contenido)

    # Filtrar la lista para excluir palabras que contengan dígitos (números)
    palabras_filtradas = [
        palabra for palabra in palabras if not any(c.isdigit() for c in palabra)
    ]

    # Calcular el número de palabras filtradas
    conteo = len(palabras_filtradas)

    # Abrir el archivo en modo append para añadir la línea con el conteo al
final
    with open(fichero, "a", encoding="utf-8") as f:
        f.write(f"\nNúmero de palabras: {conteo}")

    # Retornar el conteo de palabras
    return conteo
  except FileNotFoundError:
    # Manejar el caso en que el archivo no se encuentre
    print(f"El archivo {fichero} no se encontró.")
    return 0


# Llamada de prueba a la función con el archivo especificado
resultado = cuentaPalabras("./ficheros/econom.txt")
print(f"Conteo de palabras: {resultado}")
```

UF1844-Ejercicio01_VISUS_VICTOR.py U | UF1844-Ejercicio02_VISUS_VICTOR.py U | UF1844-Ejercicio03_VISUS_VICTOR.py U | econom.txt U

ficheros > econom.txt

```
La economía global está en crecimiento. En 2023, el PIB mundial aumentó un 3.5%. Los países desarrollados como Estados Unidos y Ale
Número de palabras: 68
```

PROBLEMAS SALIDA CONSOLA DE DEPURACIÓN TERMINAL PUERTOS GITLENS

bash

```
Victor@NEUROMANTE MINGW64 /c/AppDesarrollo/python (main)
$ py 00_examen/UF1844_VISUS_VICTOR/UF1844-Ejercicio03_VISUS_VICTOR.py
Conteo de palabras: 68

Victor@NEUROMANTE MINGW64 /c/AppDesarrollo/python (main)
$
```